

Llenguatges de programació

Memòria del projecte



Ivan González (ivan.gf@students.salle.url.edu)

Àlex Cano (alex.cano@students.salle.url.edu)

Víctor Zalabardo (victor.zalabardo@students.salle.url.edu)

Alba Mayol (alba.mayol@students.salle.url.edu)

Pol Estapé (pol.estape@students.salle.url.edu)

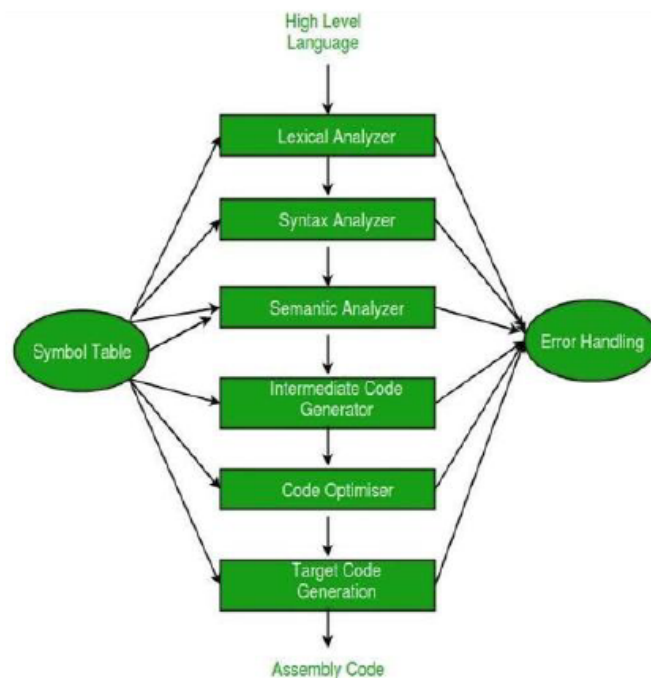
Índex

1. Introducció.....	3
2. Especificació del llenguatge.....	4
2.1 Constants Literals	4
2.2 Operadors de Comparació	4
2.3 Operadors Aritmètics	5
2.4 Operadors Lògics	5
2.5 Estructures de Control i Funcions	5
2.6 Tipus de Dades.....	6
2.7 Símbols Especials i Puntuació.....	6
2.8 Identificadors	6
2.9 Símbols Especials.....	7
3. Gramàtica.....	8
4. Mòduls del compilador.....	10
4.1. PreProcessor	10
4.2. Scanner	11
4.3. Parser.....	12
4.4. TreeAST.....	14
4.5. Anàlisi Semàntic	15
4.6. TACGenerator.....	16
4.7. MIPS.....	17
4.8. ErrorHandler	18
5. Estructures pròpies.....	20
5.1. Token	20
5.2. Node.....	21
5.3. SymbolTable.....	21
6. Metodologia utilitzada.....	23
7. Conclusions	25
8. Diagrama de classes.....	26
9. Bibliografia.....	27

1. Introducció

L'objectiu d'aquest projecte ha sigut posar en pràctica els coneixements adquirits al llarg de l'assignatura sobre el funcionament dels compiladors, tots els processos i transformacions per els que un codi passa per acabar poder essent un fitxer executable. Per fer-ho, es va decidir desenvolupar un llenguatge propi anomenat Andalus Language, creat desde zero, amb una sintaxi definida i unes regles específiques.

Pel que fa al plantejament general del projecte, aquest consisteix en dissenyar i implementar totes les fases d'un compilador, des de la definició de la gramàtica, l'anàlisi lèxic, el sintàctic, semàntic, generació de codi intermedi, generació de missatges d'error... Es per això, que havent passat per tots aquestes parts, hem après com funciona la interpretació d'un codi qualsevol per part d'un compilador. I això ha permès entendre com un ordinador pot i sap interpretar instruccions escrites en un llenguatge d'alt nivell.



Il·lustració 1: Fases d'un compilador

El resultat ha sigut un llenguatge simple i proper però sense deixar de ser complet i consistent, que ens ha permès entendre com de complexos són els llenguatges moderns i el que no, i els compiladors.

2. Especificació del llenguatge

L'objectiu del Andalus Language és que sigui un llenguatge basat en l'andalús, que sigui proper, senzill i clar, però que també tingui la seva pròpia "personalitat". Aquest llenguatge permet definir variables, estructures de control de flux, expressions i funcions, fent servir doncs símbols propis i paraules reservades que s'han establert. A continuació, es passa a aprofundir sobre els tokens i lexemes.

Tokens i lexemes

Sapiguent que un token és una unitat lèxica bàsica del llenguatge. Aquests, doncs, representen paraules clau, operadors, identificadors, etc. Cada token pot estar format per un o més lexemes, els quals són un conjunt concret de caràcters, que compleix amb el patró d'un token.

2.1 Constants Literals

Definició dels tokens que representen valors constants dins el llenguatge, els quals poden ser enters, decimals, valors lògics o caràcters individuals.

- **CTE_ENTER:** Representa tipus enter positius o negatius.
- **CTE_FLOAT:** Representa tipus decimals.
- **CTE_LOGIC:** Tokens per valors lògics.
- **CTE_CHAR:** Caràcters entre cometes simples, representa una lletra en format ASCII.

2.2 Operadors de Comparació

Els tokens d'aquest apartat reflexen operadors que comparen valors i retornen un resultat lògic (if, else,...):

- **GT (Greater Than):** Equivalent a *mah_mayor* per a ">".
- **GE (Greater or Equal):** Equivalent a *mah_mayor_o_iguah* per a ">=".
- **LT (Less Than):** Equivalent a *mah_menor* per a "<".
- **LE (Less or Equal):** Equivalent a *mah_menor_o_iguah* per a "<=".
- **NE (Not Equal):** Equivalent a *noh_loh_mismoh* per a "!=".
- **EQ (Equal // ==):** Equivalent a *loh_mismoh* per a "==".
- **EQU (Equals // =):** Equivalent a *es* per a "=".

2.3 Operadors Aritmètics

Els tokens d'aquest apartat reflexen un conjunt d'operacions matemàtiques bàsiques.

- **SUMA:** Equivalent a "+".
- **RESTA:** Equivalent a "-".
- **MULT:** Equivalent a "*".
- **DIVISION:** Equivalent a "/".
- **MOD:** Equivalent a "%".

2.4 Operadors Lògics

Els tokens d'aquest apartat reflexen operadors per a combinacions lògiques dins expressions booleans.

- **AND (&&):** Equivalent al operador "and" (lexema "y").
- **OR (| |):** Equivalent al operador "or" (lexema "o").

2.5 Estructures de Control i Funcions

Els tokens d'aquest apartat reflexen paraules reservades per a estructures bàsiques i clàssiques, de control de fluxes del programa i declaracions de funcions.

- **IF:** Equivalent a *zi_pueh_ze* per a "si".
- **ELSE:** Equivalent a *zi_nah* per a "sinó".
- **WHILE:** Equivalent a *to_er_rato_ke* per a "mentres".
- **FOR:** Equivalent a *pa_keh* per a "per a".
- **FUNC:** Equivalent a *definih* per a "funció".
- **RETURN:** Equivalent a *devolveh* per a "retornar".
- **MAIN:** Equivalent a *main* per a "funció main".

2.6 Tipus de Dades

Els tokens d'aquest apartat representen tots els tipus de dades que es poden fer servir al llenguatge.

- **SENCER:** Equivalent a *enterillo* per a "enter".
- **FLOAT:** Equivalent a *decimah* per a "decimal".
- **LOGIC:** Equivalent a *verdah_o_mentirah* per a "booleà".
- **CHAR:** Equivalent a *una_letra* per a "caràcter".
- **STRING:** Equivalent a *cadena* per a "cadena de caràcters".
- **VOID:** Equivalent a *er_vacio* per a "buit".

2.7 Símbols Especials i Puntuació

Els tokens d'aquest apartat representen signes de puntuació i símbols especials que estructuraven el codi, com són parèntesis, comes, punts i coma...

- **PO (Parèntesis Obert):** Equivalent a " (".
- **PT (Parèntesis Tancat):** Equivalent a ") ".
- **ABRE_CORCHETE (Corchete Obert):** Equivalent a " [".
- **CIERRA_CORCHETE (Corchete Tancat):** Equivalent a "] ".
- **COMA (Coma):** Equivalent a ,.
- **PUNT_COMA (Punt i Coma):** Equivalent a " ; ".
- **DOS_PUNTS (Dos Punts):** Equivalent a " : ".
- **COMILLAS (Comilles Dobles):** Equivalent a " " ".

2.8 Identificadors

Aquest token pot representar variables i/o funcions definits per l'usuari. El token corresponent és "ID" (o el nom que li volem donar) i els lexemes són els diferents identificadors.

El lexema de l'identificador no és útil en l'anàlisi sintàctica però és fonamental per l'anàlisi semàntica.

En el nostre llenguatge, els diferents noms per als identificadors permeten caràcters i símbols especials, però no números. Per exemple: *altura_* o *edat*.

2.9 Símbols Especials

- ϵ : Representa el conjunt buit. És a dir, quan una producció no consumeix cap símbol.
- $\$$: Representa el final del fitxer (EOF, End Of File). És l'axioma.

3. Gramàtica

A continuació una definició formal de la nostra gramàtica en notació BNF (Backus-Naur Form), dissenyada per descriure la sintaxi del llenguatge. Aquesta gramàtica inclou el bloc principal del programa, declaracions de funcions i variables, assignacions, expressions, estructures de control de fluxe com son condicions i bucles...

<codi> ::= <declaracio_funcio_lista> <principal> <declaracio_funcio_lista>

<declaracio_funcio_lista> ::= <declaracio_funcio> <declaracio_funcio_lista> | ε

<principal> ::= MAIN PO PT ABRE_CORCHETE <instruccions> CIERRA_CORCHETE

<instruccions> ::= <instruccio> <instruccions>

<instruccio> ::= <declaracio_variable> | <assignacio> | <expressio> PUNT_COMA | <ponderacio> |
 <bucle_for> | <devolucio> | <crida_funcio> | ε

<expressio> ::= <terme> <expressio'>

<expressio'> ::= SUMA <terme> <expressio'> | RESTA <terme> <expressio'> | ε

<terme> ::= <factor> <terme'>

<terme'> ::= MULT <factor> <terme'> | DIVISION <factor> <terme'> | MOD <factor> <terme'> | ε

<factor> ::= PO <expressio> PT | <numero> | ID | <crida_funcio>

<numero> ::= <numero_enter> | <numero_decimal>

<numero_enter> ::= CTE_ENTER

<numero_decimal> ::= CTE_FLOAT

<lletra> ::= CTE_CHAR

<cadena> ::= COMILLAS <text> COMILLAS

<text> ::= <lletra> <text> | ε

<tipus_variable> ::= SENCER | FLOAT | LOGIC | CHAR | STRING | VOID

<declaracio_variable> ::= <tipus_variable> <llista_variables> PUNT_COMA

<llista_variables> ::= <variable> <llista_variables'>

<llista_variables'> ::= COMA <variable> <llista_variables'> | ε

4. Mòduls del compilador

4.1. PreProcessor

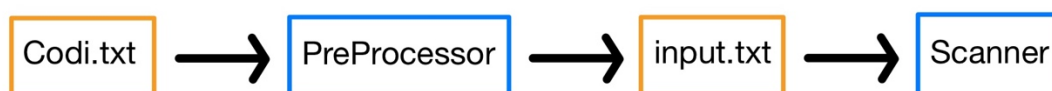
El preprocessador constitueix la primera estació del nostre recorregut de compilació. Al rebre el codi tal com el programa l'usuari, el seu paper és netejar-lo fins a deixar un flux de text homogeni, sense distraccions sintàctiques que compliquin l'anàlisi lèxic. Llavors, eliminem comentaris tant d'una com de diverses línies, esborrem espaiats i descartem aquelles línies que, després de la neteja, han quedat buides. D'aquesta manera garantim que l'única cosa que queda al seu pas és informació semànticament més important. Després, inserim espais al voltant de cada símbol de puntuació significatiu. D'aquesta separació artificial té una conseqüència immediata, el mòdul següent podrà extreure els tokens mitjançant una senzilla divisió per espais en blanc.

Una vegada normalitzat, posem el text a un fitxer intermedi, input.txt, que farà de pont permanent amb el següent mòdul. Amb aquesta estratègia parem la sortida del preprocessador i obtindrem un fitxet txt per poder poder observar i validar la neteja que s'ha fet. També, col·loquem al final de l'arxiu un caràcter per indicar el final, que és el caràcter dòlar '\$', que serveix per al parser per saber si ja ha arribat al final del codi, o per altra banda, ha hagut algun error sintàctic. Encara que sembli un detall menor, la presència d'aquest símbol simplifica la lògica de detecció d'errors i reforça la importància dels missatges que retorna el compilador al programador.

```
main ( ) {  
  enterillo num ;  
  enterillo resultado ;  
  num es 5 ;  
  resultado es num ;  
}  
$
```

Il·lustració 2: Resultat del codi netejat al input.txt

Finalment, realitzar un preprocessador separat aporta beneficis molt apreciats. Ens permet mantenir el scanner lliure de preocupacions com la gestió dels comentaris i obre la porta per ampliar el llenguatge i tot això sense tocar el nucli de l'analitzador lèxic o el parser.



Il·lustració 3: Flux del procés del compilador.

4.2. Scanner

El primer que s'ha fet és crear una classe anomenada Token per representar cada unitat lèxica (token) que el scanner identifica dins del codi font. Aquesta classe actua com a contenidor de la informació essencial de cada token, com ara el seu valor literal, el tipus i la línia on es troba. Aquesta estructura és clau perquè el parser pugui interpretar correctament el codi de l'usuari. En parlarem amb més detall a l'apartat de Estructures Pròpies.

Un cop definida la classe Token, el següent pas va ser formalitzar el diccionari de paraules clau que inicialment teníem escrit a mà (en un document), per poder fer comparacions automàtiques durant l'anàlisi del codi. Aquest conjunt inclou totes les paraules reservades del llenguatge com enterillo, si, sino, etc.

En la següent imatge podem apreciar la implementació de part del nostre diccionari.

```
palabrasClave.put("enterillo", "SENCER");
palabrasClave.put("decimah", "FLOAT");
palabrasClave.put("verdah_o_mentirah", "LOGIC");
palabrasClave.put("una_letra", "CHAR");           // tipo 'char'
palabrasClave.put("cadenica", "STRING");         // tipo 'string'
palabrasClave.put("er_vacio", "VOID");           // tipo 'void'

palabrasClave.put("mah_mayor", "GT");             // mayor que '>'
palabrasClave.put("mah_mayor_o_iguah", "GE");     // mayor o igual que '>='
palabrasClave.put("mah_menor", "LT");             // menor que '<'
palabrasClave.put("mah_menor_o_iguah", "LE");     // menor o igual que '<='
palabrasClave.put("noh_loh_mismoh", "NE");        // diferente de '!='
palabrasClave.put("loh_mismoh", "EQ");            // igual a '=='
palabrasClave.put("es", "EQU");                  // asignación '='

patrones.put("\\(", "PO"); // paréntesis abierto '('
patrones.put("\\)", "PT"); // paréntesis cerrado ')'
```

Il·lustració 4: Implementació al codi del nostre diccionari.

Finalment, es va implementar un sistema dins d'una classe anomenada Lexer que s'encarrega de llegir el codi font, identificar paraules i símbols, i generar un token per a cadascun d'aquests elements:

- Aquest sistema recorre el codi font caràcter a caràcter i decideix què fer en funció del tipus de caràcter que troba.

- Si el text llegit coincideix exactament amb una paraula reservada com if, else o enter, es compara directament i es classifica segons el diccionari.
- Si no coincideix amb cap paraula reservada, es comprova si compleix algun patró predefinit mitjançant regex. Això inclou números enters, literals de caràcter, identificadors vàlids, etc.

Aquest procés permet detectar i classificar correctament qualsevol element del codi font, i preparar-lo per ser analitzat posteriorment pel parser.

4.3. Parser

El primer que vam fer va ser intentar implementar un sistema automàtic que generés els conjunts FIRST i FOLLOW a partir de la gramàtica, així com la taula de parsing LL(1). Aquest sistema havia de llegir les produccions, calcular les derivacions i construir una taula que indicara quina regla aplicar per a cada combinació de símbol no terminal i token. Tot i això, no vam poder implementar aquest sistema automàtic.

Per aquest motiu, vam calcular a mà els conjunts FIRST i FOLLOW de cada no terminal, i també vam construir manualment la taula de parsing. Aquesta taula es va introduir en format JSON dins del projecte, de manera que el parser la pogués consultar durant l'anàlisi sintàctica.

```
"codi": {
  "SENCER": [
    "<declaracio_funcio_lista>",
    "<principal>",
    "<declaracio_funcio_lista>"
  ],
  "FLOAT": [
    "<declaracio_funcio_lista>",
    "<principal>",
    "<declaracio_funcio_lista>"
  ],
  "LOGIC": [
    "<declaracio_funcio_lista>",
    "<principal>",
    "<declaracio_funcio_lista>"
  ],
  "CHAR": [
    "<declaracio_funcio_lista>",
```

Il·lustració 5: Imatge de la nostra parsing_table.json

Un cop vam tenir la taula de parsing definida i codificada en format JSON, vam implementar un **parser tabulat** que utilitza una taula LL(1) per guiar el procés d'anàlisi sintàctica.

Aquest parser funciona mitjançant una pila i un recorregut dels tokens generats pel scanner. Els principals components d'aquest sistema són:

- **La pila de derivació**, que inicialment conté el símbol inicial de la gramàtica. A mesura que s'apliquen produccions, s'afegeixen o es treuen símbols d'aquesta pila.
- **La taula LL(1)**, que es carrega des del fitxer JSON. Aquesta taula indica quina producció s'ha d'aplicar per a cada combinació de no terminal i tipus de token. Si no hi ha cap producció per a la combinació trobada, es considera un error sintàctic.

També es va implementant un sistema de **construcció de l'arbre sintàctic**. Cada vegada que s'aplica una producció, es crea un node per al símbol no terminal i, dins d'aquest node, s'hi afegeixen fills corresponents als símbols de la producció.

Els terminals també es converteixen en fulles de l'arbre. D'aquesta manera, es genera una representació jeràrquica del codi, que servirà de base per a les fases posteriors del compilador, com la generació de codi intermedi o final.

```
<instruccio>
  <instruccio_id>
    <instruccio_id'>
      ;
      <expressio>
        <expressio'>
          ε
          <terme>
            <terme'>
              ε
              <factor>
                <factor'>
                  ε
                  kk
                es
              resultado
```

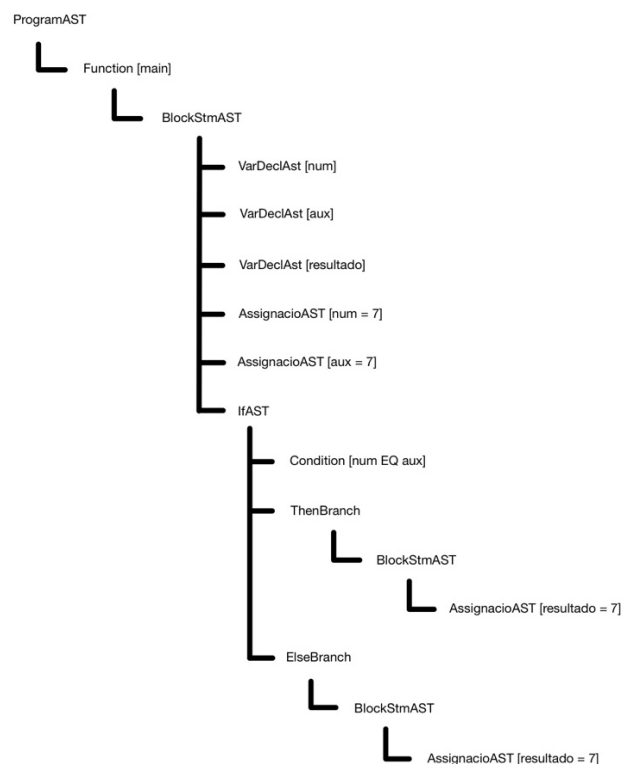
Il·lustració 6: Imatge del nostre Parser Tree per la terminal.

4.4. TreeAST

Quan finalitza l'etapa del parser, disposem d'un arbre de sintaxi concret, el parser tree, la forma de la qual reflecteix cada regla de la nostra gramàtica. Aquest arbre és perfecte per poder verificar que el programa compleix la sintaxi, però resulta poc pràctica per a les fases que venen després. Per aquesta raó convertim el nostre parser tree en un arbre de sintaxi abstracta (AST). Una representació més compacta que conserva l'essència gramatical.

La transformació del nostre parser tree al arbre AST es realitza dins de la classe TreeAST, que rep l'arrel del parser tree i la recorre de dalt a baix. Llavors, per a cada node gramatical invoquem un mètode especialitzat buildIf, buildFor, buildAssignment, etc. Aquests mètodes examinen els fills del node, treuen els parèntesi, claus o símbols de tancament i creen l'objecte AST que millor reflecteix la construcció original. El procediment és recursiu, al construir, per exemple, un IfAST cridem a buildExpression per a sintetitzar la condició i a buildBlock per a crear el cos. També, cada crida retorna al seu torn objectes ja abstractes, de manera que la conversió va filtrant l'estructura fins que tot el sub-arbre queda en forma AST.

Els nodes es dissenyen seguint una lògica semàntica clara. Comptem amb classes de sentència com AssignacioAST, IfAST, ForAST, ReturnStmAST, etc. I també amb classes d'expressió com BinaryExprAST, VariableExpAST, CallExpresioAST, entre altres. Llavors, cada tipus encapsula exactament la informació que serà rellevant més endavant com per exemple, identificadors, operadors, valors literals i la línia d'origen per informar dels errors semàntics. Tots els camps es fixen en el constructor i només s'exposen a través de getters, de manera que l'arbre resulta immutable i qualsevol pas posterior pot recórrer-lo sense risc d'interferències.



Il·lustració 7: Representació del nostre arbre AST.

Aquest disseny aporta diferents avantatges decisius. Primer, desacobla la resta del compilador dels detalls de la gramàtica concreta. Llavors, si demà canviem les regles per a fer la gramàtica LL(1), bastarà amb actualitzar el TreeAST i això farà que les fases següents ni s'assabentaran. Segon, simplifica els anàlisis. A l'haver eliminat soroll, cada recorregut es concentra en la informació semàntica pura i redueix la seva complexitat algorítmica.

Finalment, disposar del AST ens permet realitzar la fase de l'anàlisi semàntic amb garanties. Llavors, amb el arbre AST que hem construït, construïm la taula de símbols, comprovem tipus i detectem usos incorrectes de variables. També, gràcies al arbre AST, podem generar el TAC una vegada s'hagi comprovat correctament la semàntica, on expressarà cada operació en instruccions.

```
Program
Function main() : VOID
{
    VarDecl SENCER num
    VarDecl SENCER aux
    VarDecl SENCER resultado
    Assign num = 7
    Assign aux = 7
    If (num EQ aux)
    {
        Assign resultado = 5
    }
    Else
    {
        Assign resultado = 1
    }
}
```

Il·lustració 8: Representació del nostre AST per la terminal.

4.5. Anàlisi Semàntic

Un cop tenim l'AST construït, podem començar a validar que el programa té sentit a nivell semàntic, no només sintàctic. Aquesta fase comprova coses com que les variables estiguin declarades abans d'utilitzar-se, que no es repeteixin noms dins el mateix àmbit, que les assignacions tinguin tipus compatibles o que les funcions rebin exactament els paràmetres que toquen. Tot això ho fem amb dues passes principals. Una per recollir tota la informació necessària i una altra per validar-la.

La primera passada es fa amb la classe SymbolCollector. Aquesta s'encarrega de recórrer l'arbre i construir la taula de símbols, tenint en compte els àmbits (scopes). Per cada funció, entra dins del seu àmbit, registra els paràmetres (comprovant que no es repeteixin noms) i després processa tot el seu cos. Les variables només es poden declarar una vegada dins del mateix àmbit, i si ja existeix una amb

el mateix nom, llencem error. També gestionem l'offset de cada variable per preparar la generació de codi posterior. Si una variable es declara amb un valor inicial, ja queda marcada com inicialitzada.

Un cop tenim la taula de símbols completa, passem a fer la comprovació semàntica pròpiament dita amb SemanticChecker. Aquí validem els tipus, com ara que no es pugui assignar un valor d'un tipus a una variable d'un altre, o que no es faci un return amb valor en una funció void (ni a l'inrevés). També comprovem que totes les rutes d'una funció no-void acabin amb un return, i que les condicions dins d'un if o un for siguin de tipus lògic.

A més, quan es crida una funció, es comprova que existeixi i que se li passin exactament els arguments esperats, ni un més ni un menys, i amb els tipus correctes. També es valida que qualsevol variable que s'utilitza estigui inicialitzada prèviament; no permetem llegir d'una variable que encara no té valor.

Per gestionar tot això correctament, es fa servir un sistema jeràrquic d'àmbits amb la classe ScopeManager, que manté una pila amb el nom dels scopes actius i permet buscar símbols des del més proper al més llunyà. Això assegura que si es declara una variable amb el mateix nom que una d'un àmbit exterior, s'utilitza la més propera (la més interna), tal com toca. També impedeix redeclarar una variable dins el mateix bloc.

Finalment, destacar que no permetem la reinicialització de variables, però sí la reassignació (sempre que siguin del mateix tipus). Això, juntament amb la prohibició d'usar variables no inicialitzades, ens ha ajudat a detectar molts errors comuns durant les proves. Tampoc permetem variables globals, només globals poden ser les funcions, però sí que es poden fer crides recursives. A nivell de tipus, tot és més senzill perquè només treballem amb enters, així que tot es resumeix a validar que sigui enter o que una funció void no retorni res. Aquest sistema d'anàlisi semàntic ens ha garantit que tots els programes que passen aquesta fase estan ben formats a nivell de significat, i prepara el terreny per generar el codi intermedi sense sorpreses.

4.6. TACGenerator

Un cop enllestit l'arbre AST (Abstract Syntax Tree) ja es té tot el necessari per començar a generar el codi intermedi. De fet, amb l'arbre de parsing fet (i considerant que no hi ha errors de cap tipus) ja es podria començar a picar codi intermedi. Però un arbre AST facilita molt la feina i permet tenir un codi molt més net i endreçat.

En el nostre cas s'ha decidit implementar el llenguatge intermedi anomenat TAC (Three-Adressed Code). La gràcia del TAC, o allò que el fa especial, és que tracta de convertir el codi font en un codi que conté, com a màxim, tres adreces per línia (per instrucció). Aquesta decisió parteix de dos arguments. El primer és que aquest és el llenguatge que s'ha vist i ensenyat a classe, fent-lo el més senzill i ràpid d'implementar. La segona raó d'escollir-lo és que té un nivell d'abstracció mitjà, és a dir, que no queda ni molt lluny ni molt a prop del codi màquina. D'aquesta manera no ens compliquem massa la vida i a la vegada queda un codi molt assequible i comprensible tant per l'optimitzador com pel generador de codi màquina.

Un cop escollit el codi intermedi a implementar, s'ha tractat de planificar quines classes s'usarien. S'han escollit classes distintives per a les següents funcionalitats: assignacions, operacions (que han de ser binàries per mantenir els requeriments del TAC), salts de línia, salts de línia condicionals, etiquetes i retorns. D'aquesta manera s'aconsegueix crear un codi organitzat i a la vegada més senzill d'entendre i manipular.

```
func_main:
i = 0
aux = 0
t0 = i > 0
ifFalse t0 goto L0
aa = 3
goto L1
L0:
L1:
future_param i
future_param aux
call fibonacci
t1 = RET
i = t1
endfunc_main:
```

Arribats en aquest punt ja es pot començar a crear el codi intermedi. Per a fer-ho s'ha creat una classe principal anomenada "TACGenerator.java". Aquesta classe s'encarrega de recórrer l'AST i anar generant tota la seqüència d'instruccions que constituïran el TAC. D'aquesta forma, s'identifica quina funció té cada element que s'obté al recórrer l'AST i es va generant, línia per línia, el codi intermedi. Per a consultar que el codi agafa la forma correcta s'ha fet ús de la classe "TACEmitter.java".

Il·lustració 9: Exemple de codi TAC

Finalment, s'ha creat un optimitzador de codi intermedi. Aquest tracta de fer el TAC més eficient aplicant les tècniques més comunes, com ara la simplificació de constants, propagació de còpies o eliminació de variables temporals. Tot i així, no s'ha arribat a implementar un optimitzador plenament funcional i s'ha deixat pendent de finalitzar. Tot i així, s'ha aprofitat aquest optimitzador per comptar el nombre de variables de cada funció, fet que ajudarà després a construir el MIPS.

4.7. MIPS

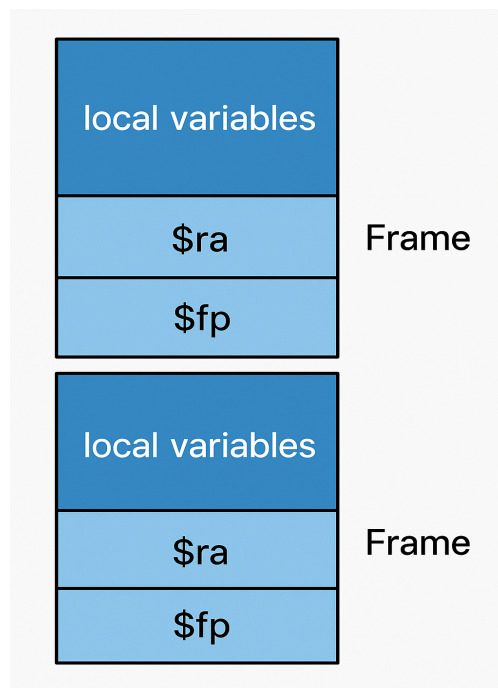
Amb el codi TAC ja tindríem la lògica del nostre programa, ara tocava transformar aquest codi a codi MIPS. El primer que vam fer va ser traduir línia a línia. Vam valorar quines línies retornava el TAC i quina seria la seva traducció en MIPS.

Per fer-ho, vam analitzar cada tipus d'instrucció TAC de forma individual: assignacions, operacions aritmètiques, salts incondicionals, salts condicionals i etiquetes. Un cop identificat el tipus, vam escriure manualment la seva traducció a MIPS. Per exemple, una suma com $a = b + c$ es transforma en una seqüència d'instruccions MIPS que carrega els valors i realitza l'operació amb add. També vam definir com representar els salts (goto, if) i les etiquetes de control de flux, utilitzant instruccions com j, beq, bne, i label: per mantenir el comportament del programa original. Aquest procés ens va permetre construir un traductor capaç de convertir el codi TAC a un conjunt d'instruccions MIPS funcionals i equivalents.

Quan ja teníem el codi MIPS funcional, ens vam adonar d'un problema important: no hi havia prou registres per emmagatzemar totes les variables temporals necessàries durant l'execució. Per solucionar-ho, vam implementar un sistema de frames. Cada frame representa l'espai de treball d'una funció i s'encarrega de guardar les seves pròpies variables temporals, així com els registres especials

\$ra (return address) i \$fp (frame pointer), que són essencials per a la gestió de les crides i retorns de funció.

Finalment, per calcular la mida de cada frame, vam analitzar quantes variables temporals diferents apareixien al codi TAC de cada mètode. Això ens va permetre reservar just l'espai necessari per a cada funció i mantenir un accés ordenat i segur a les seves dades durant l'execució.



Il·lustració 10: Representació de la nostre memòria.

4.8. ErrorHandler

L'ErrorHandler és la classe encarregada de gestionar tots els errors que poden sorgir durant les fases d'anàlisi lèxica, sintàctica i semàntica. Funciona com un *singleton*, és a dir, només hi ha una instància d'aquesta classe durant tota l'execució del compilador, i la resta de classes hi accedeixen quan necessiten registrar algun error. Això permet tenir tots els errors centralitzats i organitzats en un sol lloc.

Cada vegada que es detecta un error, s'afegeix a una llista interna amb informació sobre la línia on ha ocorregut, el tipus d'error (lèxic, sintàctic o semàntic) i un missatge descriptiu. Si durant l'anàlisi es troba almenys un error, no es continua amb la generació de codi TAC ni MIPS.

La classe també disposa de mètodes específics per a errors semàntics típics, com ara si es repeteix un paràmetre en una funció, si s'utilitza una variable que no ha estat definida o inicialitzada, si es redefineix una funció, etc.

Al final del procés, si hi ha errors, s'imprimeixen tots per consola perquè l'usuari els pugui revisar fàcilment. En cas contrari, s'indica que no s'han trobat errors. Aquesta classe fa que la detecció i gestió d'errors sigui molt més neta i que l'usuari rebi una retroalimentació clara i útil.

```
No se han encontrado errores léxicos
Errores detectados:
[Semántico] Línea 26: Número de argumentos incorrecto
[Semántico] Línea 27: Variable 'kk' usada sin inicializar
[Semántico] Línea 0: La función fibonacci debe devolver un valor
[Semántico] Línea 14: return no lleva valor
```

Il·lustració 11: Exemple d'error de codi

5. Estructures pròpies

5.1. Token

En el procés de construcció d'un compilador, un dels primers passos fonamentals és l'anàlisi lèxica, on el codi font escrit per l'usuari es divideix en unitats lèxiques conegudes com a tokens. Un token és una estructura que representa una peça significativa del text d'entrada, com ara una paraula clau, un identificador, un nombre, un operador, etc.

En el nostre llenguatge, hem definit el token com una classe pròpia amb tres atributs principals:

- **Tipus:** Indica el tipus de token que és, com per exemple si es tracta d'un identificador, una paraula clau, un operador, un símbol de puntuació, etc. Aquesta informació és essencial per a la fase d'anàlisi sintàctica.
- **Valor:** Conté el text literal que s'ha extret directament de l'entrada de l'usuari. És a dir, la cadena original que ha generat aquest token.
- **Linia:** Representa el número de línia del codi font on s'ha trobat el token. Això és molt útil per proporcionar missatges d'error clars i precisos durant la compilació.

Aquesta estructura ens permet gestionar de manera clara i estructurada tota la informació necessària sobre cada unitat lèxica identificada pel nostre analitzador.

```
public class Token {  
    3 usages  
    private String tipus;  
    3 usages  
    private String valor;  
    2 usages  
    private int linea;  
}
```

Il·lustració 11: Imatge del nostre codi.

5.2. Node

Durant la fase d'anàlisi sintàctica del compilador, es construeix un arbre que representa l'estructura jeràrquica del codi font segons la gramàtica del llenguatge. Per representar aquest arbre sintàctic, hem creat una estructura pròpia anomenada Node.

La classe Node encapsula la informació necessària per formar aquest arbre i té els següents atributs:

- **Token:** És un objecte de la classe Token que representa el símbol gramatical (terminal o no terminal) associat a aquest node. Permet identificar quina part de la gramàtica representa.
- **Fills:** És una llista d'objectes Node que conté els nodes fills d'aquest node dins de l'arbre. Aquesta estructura recursiva reflecteix la jerarquia del llenguatge segons les regles de producció.
- **Usado:** És un atribut booleà que indica si aquest node ha estat utilitzat en alguna part del procés d'anàlisi o generació.

Aquesta estructura ens permet construir, recórrer i manipular l'arbre sintàctic amb facilitat, i és clau per a etapes posteriors com la generació de codi o l'anàlisi semàntic.

```
public class Node {  
    4 usages  
    Token token;  
    6 usages  
    List<Node> fills;  
    3 usages  
    boolean usado;  
}
```

Il·lustració 12: Imatge del nostre codi.

5.3. SymbolTable

Per gestionar tota la informació relacionada amb les declaracions dins del nostre llenguatge (variables, funcions, etc.), hem implementat una estructura pròpia anomenada SymbolTable. Aquesta taula de símbols és l'encarregada de registrar cada element declarat pel programador juntament amb informació clau com el seu nom, el tipus i l'àmbit en què ha estat definit.

A més, com que en qualsevol llenguatge amb blocs o funcions els àmbits (scopes) són essencials, hem creat una altra classe que treballa conjuntament amb la taula: el `ScopeManager`. Aquesta classe s'encarrega de mantenir controlats els canvis d'àmbit, cosa que passa cada vegada que entrem o sortim d'un bloc de codi, una funció o qualsevol estructura que defineixi un nou context.

A la pràctica, cada vegada que entrem en un nou àmbit, es crea un identificador únic (com per exemple `scope3`, `scope4`, etc.) i s'apila a una pila de scopes actius. Això ens permet saber en tot moment on estem, i quan sortim del bloc, fem pop de la pila i l'àmbit queda tancat.

Per representar els símbols de forma clara, hem creat una jerarquia de classes senzilla:

- ◆ **SymbolInfo**: És la base i representa qualsevol símbol, amb tres dades bàsiques: el nom, el tipus i l'àmbit on ha estat declarat.
- ◆ **SymbolInfoVariable**: Hereta de la classe anterior i afegeix informació útil per les variables com si és paràmetre, si està inicialitzada i l'offset (útil per a la generació de codi en MIPS).
- ◆ **SymbolInfoFunction**: També hereta de `SymbolInfo` i afegeix la llista de paràmetres de la funció. També guardem si té o no un return, per poder validar que el flux del programa és correcte.

D'altra banda, el `ScopeManager` s'encarrega de:

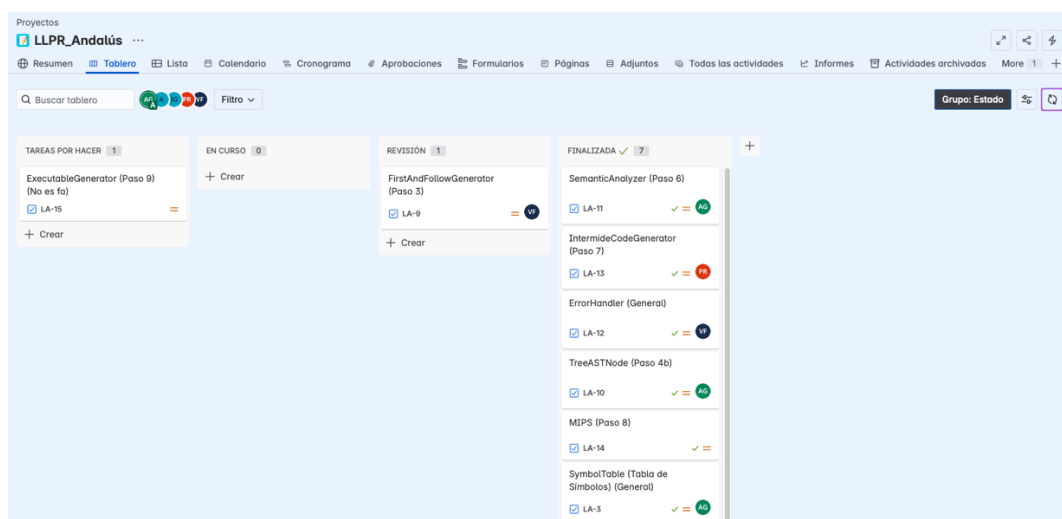
- Controlar la pila d'àmbits actius (amb noms com `scope1`, `scope2`...).
- Calcular els offsets de variables locals i paràmetres per a cada àmbit.
- Comprovar si un símbol està definit en el context actual o en algun àmbit superior.
- Retornar el símbol més proper accessible des del punt actual (per evitar confusions amb ombrejat de variables).
- Gestionar les funcions que contenen retorn, per assegurar que totes les definicions són correctes.

Tot aquest sistema treballa juntament amb l'`ErrorHandler`, de manera que si el programador intenta declarar una variable repetida dins del mateix àmbit, o accedir a una que no existeix o no està inicialitzada, es llança un error informatiu indicant exactament la línia i el problema detectat.

6. Metodologia utilitzada

Des del primer dia vam voler que l'organització del treball sigués molt constant i que tothom pogués tenir context de la situació, de manera que vam crear un repositori al GitHub. Aquest repositori és el que ens ajuda a sincronitzar i ajuntar el codi que cadascú de nostres anava creant durant el projecte. Dins d'aquest repositori, tenim la branca de referència 'dev', que era la branca origen del nostre projecte on anàvem ajuntant el codi final revisat de cada branca que es creava.

En paral·lel vam crear un espai al Jira dedicat exclusivament al projecte. Aquí teníem un tauler on vam crear tasques relacionades al projecte marcades amb números segons el pas del projecte que tocava realitzar per tenir un context i ordre de realització del projecte entre tots nosaltres. Llavors, cada persona del grup s'assignava a una de les tasques que volia realitzar i es podia anar seguint el context d'aquella tasca dins del tauler amb les columnes Tareas por hacer, En curso, Revisión i Finalizada. Finalment, dins de cada tasca s'explicava quin era el objectiu i finalitat de realitzar aquell pas del projecte, i sempre que estava en revisió, altres membres del grup accedien dins d'aquella branca per fer testing i comprovar el correcte funcionament per ajuntar a la branca 'dev' del projecte.



Il·lustració 13: Imatge del nostre tauler del Jira.

La organització entre nosaltres era una mica complicat alhora de poder veurens físicament. Perquè dins del grup som de cursos diferents i només ens veiem a les hores de classe. Per això, també vam crear un canal al Discord per fer reunions online poder parlar i debatre les diferents realitzacions del projecte.



Il·lustració 14: Imatge del nostre recorregut al GitHub.

Gràcies a aquesta combinació de GitHub, Jira i la metodologia àgil, tots vam mantenir un ordre i un context clar. El repositori mostra l'evolució tècnica del projecte i el tauler mostra l'estat funcional. Així avançàvem de manera constant, detectant els errors ràpid i aconseguint que tots nostres puguem participar de manera clara i correcte de la millor manera possible.

7. Conclusions

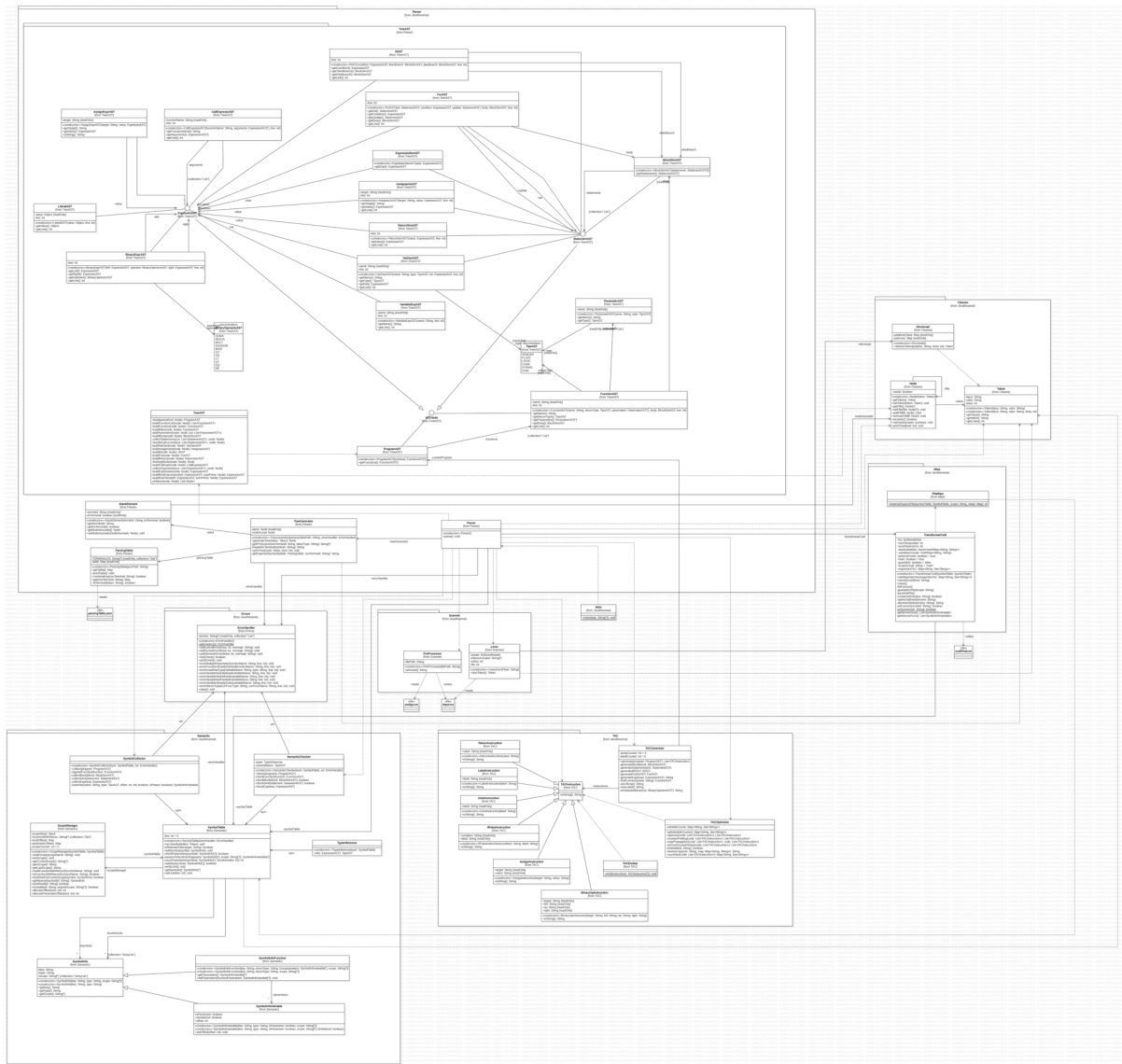
Com a conclusió d'aquest projecte, abans de res, ha sigut una experiència d'aprenentatge molt intèns. D'una banda, hem pogut veure tota la teoria vista a classe, des de la gramàtica fins a la generació de codi MIPS, amb la oportunitat de posar en pràctica els nostres coneixements. Llavors, poder completar el compilador ens va permetre comprovar de primera mà com encaixen, pas a pas, dins d'un sistema real. Cada mòdul que desenvolupem ens va obligar a revisar i buscar com implementar cada pas. Gràcies a això ara entenem la importància que té cada mòdul dins del compilador i perquè son tant dependents un dels altres.

El més complicat va arribar quan vam haver de fer les últimes parts del compilador. Vam haver de crear l'arbre AST, generar el codi TAC i després traduir-lo a MIPS. Això ens va plantejar problemes de disseny i de programació que mai havíem vist. Per a construir el AST, primer vam haver de netejar la gramàtica i treure qualsevol ambigüitat. Si deixàvem un error, es propagava i acabava bloquejant el TAC o l'anàlisi semàntica. Al treballar amb el TAC vam aprendre que era fonamental controlar bé les variables temporals i les etiquetes. Finalment, al passar-ho al MIPS vam entendre l'important que és utilitzar bé els registres i les piles d'activació sense malgastar recursos. Van ser passos llargs, però al final vam poder implementar tot això i poder aprendre la importància que tenen dins del compilador.

Un aprenentatge clau va ser adonar-nos de l'important que és tenir una bona gramàtica des del principi. Cada vegada que milloràvem les regles LL(1), l'analitzador sintàctic es feia més senzill i, també, era més fàcil crear el AST i fer les comprovacions semàntiques. Al veure aquest efecte que tenia dins del projecte, vam entendre que invertir temps a definir bé la gramàtica era un dels passos més importants per tenir un bon compilador.

Finalment, el projecte que hem realitzat ens convens i ara entenem com s'organitza i funciona un compilador modern. Des de la neteja del codi font, fins a la traducció a assemblador. També, hem pogut aplicar els coneixements teòrics que hem vist a classe i d'aquesta manera aprendre més posant-ho en pràctica. El treball en equip ha sigut molt important per poder crear un projecte com aquest, quan algú de nostres no entenia alguna cosa, entre nostres ens explicàvem i apreníem conjuntament per poder tirar endavant el compilador.

8. Diagrama de classes



Il·lustració 15: Diagrama de classes del projecte.

9. Bibliografía

- Colaboradores de Wikipedia. (2024, 12 diciembre). Árbol de sintaxis abstracta. Wikipedia, la Enciclopedia Libre. https://es.wikipedia.org/wiki/%C3%81rbol_de_sintaxis_abstracta
- Tutorialspoint. (2025, 25 marzo). *Lexical Tokens in Compiler Design*. https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_tokens.htm
- GeeksforGeeks. (2025, 25 abril). *FIRST and FOLLOW in Compiler Design*. GeeksforGeeks. <https://www.geeksforgeeks.org/why-first-and-follow-in-compiler-design/>
- Árboles de análisis abstracto | PL 22/23. (s. f.). <https://ull-esit-gradoii-pl.github.io/temas/syntax-analysis/ast.html>
- Colaboradores de Wikipedia. (2022, 27 marzo). Tabla de símbolos (compilador). Wikipedia, la Enciclopedia Libre. [https://es.wikipedia.org/wiki/Tabla_de_s%C3%ADmbolos_\(compilador\)](https://es.wikipedia.org/wiki/Tabla_de_s%C3%ADmbolos_(compilador))
- GeeksforGeeks. (2024, 27 de diciembre). *Three address code in compiler*. <https://www.geeksforgeeks.org/three-address-code-compiler/>
- GeeksforGeeks. (2025, 25 enero). *Symbol Table in Compiler*. GeeksforGeeks. <https://www.geeksforgeeks.org/symbol-table-compiler/>
- Wikipedia contributors. (2025, 14 febrero). *Parsing*. Wikipedia. <https://en.wikipedia.org/wiki/Parsing>
- Wikipedia contributors. (2025a, enero 31). *MIPS architecture*. Wikipedia. https://en.wikipedia.org/wiki/MIPS_architecture
- Colaboradores de Wikipedia. (2023, 28 mayo). *Gramática (autómata)*. Wikipedia, la Enciclopedia Libre. [https://es.wikipedia.org/wiki/Gram%C3%A1tica_\(aut%C3%B3mata\)](https://es.wikipedia.org/wiki/Gram%C3%A1tica_(aut%C3%B3mata))